

Python 变量与流程控制



学习目标：

1. 掌握 Python 变量的定义、命名规则与使用方法。
2. 理解 Python 基本数据类型（整数、浮点数、字符串、布尔值等）。
3. 掌握输入输出函数（input /print）的基本使用。
4. 熟练运用条件判断语句（if、elif、else）实现程序分支逻辑。
5. 掌握 while 循环与 for 循环的语法与使用场景。
6. 学会使用 break、continue 控制循环执行流程。
7. 能够编写简单的顺序、分支、循环结构程序。

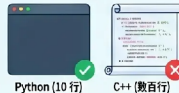
Python 编程入门：AI 时代的“指挥官”语言

本指南涵盖了 Python 的语言特性、其在 AI 领域的统治地位，以及变量、缩进和流程控制等核心基础知识。通过“指挥官”比喻，阐明了 Python 如何在保持简洁语法的同时，调用底层高性能计算资源。

为什么 Python 是 AI 开发的灵魂？



开发效率极高，代码量极少
同样的 AI 逻辑，Python 只需 10 行代码，而 C++ 可能需要数百行。



扮演“指挥官”角色
Python 负责高层逻辑，繁重的计算则由底层的 C/C++ “外挂”完成。

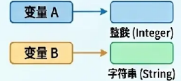
拥有全球顶尖的 AI 生态
集成了 PyTorch、TensorFlow 和 NumPy 等由全球顶尖机构维护的开源库。

Python 应用领域与代表性工具



Python 编程的核心基石

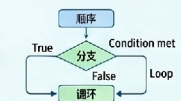
动态类型的“标签盒子”
变量是数据的名字或引用，其类型取决于指向的值且可随时改变。



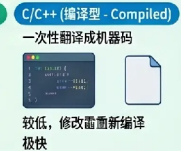
强制缩进决定代码结构
使用 4 个空格缩进代码的花括号，这种强制规范确保了代码的高度可读性。



三大流程控制结构
程序通过顺序、分支 (if-else) 和循环 (for/while) 来管理执行逻辑。



Python vs. C/C++ 执行机制对比



NotebookLM

第一章：Python 语言基础与开发环境搭建

1. Python 语言简介

1.1 什么是 Python?

Python 是一种**高层次、解释型**（运行时会自动编译为字节码）、**交互式、面向对象**的脚本语言。

它的语法设计极具可读性，使用缩进表示代码块，大量使用英文关键字而非标点符号。

Python 是**解释型语言**，开发者不需要手动编译，可以直接运行源代码。

它全面支持面向对象编程，同时也支持函数式编程。

由于语法简单、学习曲线平缓，Python 非常适合初学者，同时也能用于大型专业项目开发。

1.2 Python 的由来

Python 由荷兰程序员 **吉多·范罗苏姆 (Guido van Rossum)** 于 1989 年圣诞节期间开始设计，1991 年发布了第一个版本。

名字来源于他喜欢的英国喜剧团体 **Monty Python**（蒙提·派森），而不是蛇。



1.3 Python 核心特点

特点	说明
简单易学	语法接近自然语言，新手上手快
丰富的库和框架	拥有庞大的标准库和AI生态（NumPy、PyTorch等）
解释执行	无需编译即可运行
面向对象	支持封装、继承、多态
可扩展性强	可与 C/C++、Java 等语言集成
跨平台	Windows、Linux、macOS均可运行
社区活跃	拥有全球最大开源社区支持

1.4 Python 应用领域

领域	常用库/框架	例子
人工智能 & 机器学习	TensorFlow, PyTorch, scikit-learn	人脸识别、大语言模型（如 ChatGPT）
数据分析 & 可视化	NumPy, Pandas, Matplotlib	处理 Excel、生成图表
网站开发	Django, Flask	开发网站后端（如 Instagram、知乎）
自动化脚本	标准库	批量处理文件、自动发送邮件
爬虫	Requests, BeautifulSoup	抓取网页数据
科学计算	SciPy, SymPy	数学建模、物理模拟
游戏开发	Pygame	制作简单游戏

[🔗 点击访问Python 官网](#)

2. 为什么 AI 开发首选 Python

2.1 开发效率高、代码量少

下面这段代码展示了 Python 做 AI 有多简洁（即使你现在看不懂细节也没关系，感受一下代码行数）。

```
python
1  import numpy as np
2  from sklearn.linear_model import LinearRegression
3
4  # 准备数据 (X 是面积, y 是房价)
5  X = np.array([[50], [60], [80], [100]])
6  y = np.array([150, 180, 240, 300])
7
8  # 创建模型并训练
9  model = LinearRegression()
10 model.fit(X, y)
11
12 # 预测 75 平米的房价
13 pred = model.predict([[75]])
14 print(pred)    # 输出约 225
```

如果让你用 C++ 从头实现线性回归，需要写几百行代码（包括矩阵求逆、梯度下降等）。而 Python 只需 10 行。

2.2 海量的 AI 库与框架（核心竞争力）

领域	常用库	作用
数值计算	NumPy, SciPy	数组、矩阵、线性代数、傅里叶变换
数据处理	Pandas	处理表格数据（像 Excel 但更强大）
机器学习	scikit-learn	分类、回归、聚类等经典算法
深度学习	TensorFlow, PyTorch, Keras	搭建神经网络，训练大模型
自然语言处理	Transformers (Hugging Face)	使用 BERT、GPT 等预训练模型
计算机视觉	OpenCV, Pillow	图像处理、人脸检测
可视化	Matplotlib, Seaborn	画损失曲线、混淆矩阵、数据分布

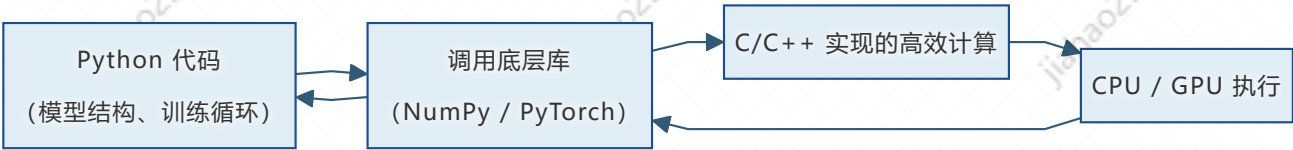
关键点：这些库大多是 **开源** 的，而且由全球顶尖机构维护（Google、Facebook、MIT 等）。你用 PyTorch 一行 `model.train()` 就能启动训练，而不用自己写几百行反向传播代码。

2.3 高性能的“外挂”：C/C++ 加速

Python 本身运行速度不如 C++，但 AI 计算的核心（矩阵乘法、卷积）被封装在底层 C/C++ 库中。

- **NumPy** 的数组运算用 C 实现，比纯 Python 循环快 50 倍以上。
- **TensorFlow/PyTorch** 的核心是 C++，并且能调用 GPU（通过 CUDA）。

程序员用 Python 写高层逻辑（模型结构、训练流程），而繁重的计算由底层的 C/C++ 代码完成。**Python 扮演“指挥官”角色，而不是“苦力”。**



2.4 庞大的社区与学习资源

AI 发展极快，新论文、新模型层出不穷。Python 社区有：

- **Stack Overflow** 上百万个 Python AI 相关问答。
 - **GitHub** 上几乎所有 AI 开源项目都是用 Python 写的。
 - **主流框架** (Hugging Face、LangChain、OpenAI API) 均以 Python 为主
- 如果你遇到问题，几乎肯定能在网上找到解决方案。

3. 搭建 Python 开发环境 (Anaconda + PyCharm)

3.1 什么是 Anaconda

Anaconda 是一个 Python 发行版，它包含：

- **Python 解释器** (即 Python 本身)
- **Conda**：一个强大的包管理和环境管理命令行工具
- **250+ 预装的数据科学库**：包括 NumPy、Pandas、Matplotlib、Jupyter Notebook 等
- **Anaconda Navigator**：图形界面，帮你点点鼠标管理环境和包

你可以把 Anaconda 理解为“**为 AI 和数据科学定制的 Python 全家桶**”。

对比原生 Python：

- 原生 Python：只包含解释器，需要你手动装每个库，环境隔离需要用 **venv** 或 **virtualenv**，安装带 C 依赖的库（如 numpy）可能还需要编译工具。
- Anaconda：自带大量库，用 **conda** 命令一键创建环境、安装包，自动解决底层依赖（甚至能装 CUDA 驱动）。

对于 AI 初学者，Anaconda 是 **最省心、最稳妥** 的选择。

3.2 安装 Anaconda

● 3.2.1 下载安装包

1. 打开浏览器，访问 Anaconda 官网：<https://www.anaconda.com/download>

2. 点击 **Download** 按钮。网站会自动检测你的操作系统 (Windows / macOS / Linux)。
3. 推荐下载 **64-Bit Graphical Installer** (图形化安装包, 约 500~600 MB)。
 - Windows: **.exe** 文件
 - macOS: **.pkg** 或 **.sh** 文件 (Apple Silicon 请选对应版本)
 - Linux: **.sh** 脚本
4. 等待下载完成。



如果官网下载慢, 也可以使用清华大学开源镜像站:

<https://mirrors.tuna.tsinghua.edu.cn/anaconda/archive/>, 选择最新的 **Anaconda3-xxxx-Windows-x86_64.exe** 等。

● 3.2.1 安装步骤 (以 Windows 为例)

1. **双击下载的安装包**, 点击「Next」。
2. 阅读许可协议, 点击「I Agree」。
3. 选择安装类型:
 - 「Just Me」(仅当前用户) → 推荐, 不需要管理员权限。
 - 「All Users」(所有用户) → 需要管理员权限, 一般不选。
4. **选择安装路径** (重要):
 - 建议保持默认: **C:\Users\你的用户名\anaconda3**
 - 或者改到其他盘, 但 **路径中不能有中文或空格** (如 **D:\Anaconda3** 是允许的, **D:\我的软件** 不行)。

5. 高级安装选项（关键一步！）：

- 勾选「Add Anaconda to my PATH environment variable」（将 Anaconda 添加到系统环境变量）。这样以后在命令行就能直接使用 `conda` 命令。
- 勾选「Register Anaconda as my default Python」（将 Anaconda 设为默认 Python 环境）。
- 点击「Install」。

6. 等待安装进度条走完（约 5~10 分钟）。安装过程中不要关闭窗口。

7. 点击「Next」，然后点击「Finish」完成安装。

● 3.2.3 验证安装

1. 按下 `Win + R`，输入 `cmd` 回车，打开命令提示符。

2. 输入 `conda --version` 并回车。

- 如果显示类似 `conda 24.1.2`，说明安装成功。
- 如果提示 `'conda' 不是内部或外部命令`，说明没有勾选 PATH，请重新运行安装程序并勾选，或者使用「Anaconda Prompt」（开始菜单中可以找到）来代替普通 `cmd`。

3. 输入 `python --version`，应显示 Anaconda 自带的 Python 版本，例如 `Python 3.11.7`。

4. 输入 `conda env list`，可以看到一个名为 `base` 的环境，前面有 `*` 号表示当前激活的环境。

3.3 用 Conda 管理虚拟环境

我们不会直接在 `base` 环境里装项目需要的库，而是为每个项目创建独立环境。

● 3.3.1 创建新环境

语法：`conda create -n 环境名 python=版本号`

● 3.3.2 激活与退出环境

激活环境: `conda activate 环境名`

激活后, 命令行提示符前面会出现 `(ai_env)`, 表示现在正处于该环境中。

退出当前环境: `conda deactivate`

● 3.3.3 查看所有环境

语法: `conda env list`

带 `*` 的是当前激活的环境。

● 3.3.4 删除环境

语法: `conda remove -n 环境名 --all`

注意: 删除后不可恢复, 请确认不需要了再删除。

3.4 用 Conda 安装和管理包

在激活的虚拟环境中, 我们可以用 `conda install` 安装所需的 Python 包。

● 3.4.1 安装包

```
python
1 # 激活环境 (确保当前在 ai_env 中)
2 conda activate ai_env
3
4 # 安装 numpy
5 conda install numpy
6
7 # 同时安装多个包
8 conda install numpy pandas matplotlib
9
10 # 安装指定版本
11 conda install numpy=1.24.3
```

Conda 会自动分析这些包之间的依赖关系，并询问你是否继续。输入 `y` 即可。

● 3.4.2 查看当前环境已安装的包

```
python
1 | conda list
```

这会列出包名、版本和来源频道。

● 3.4.3 卸载包

```
python
1 | conda remove 包名
```

● 3.4.4 更新包

```
python
1 | conda update 包名           # 更新指定包
2 | conda update --all          # 更新当前环境所有包
```

| 3.5 安装 PyCharm 并配置 Conda 环境

● 3.5.1 什么是 PyCharm

PyCharm 是由捷克公司 JetBrains 开发的专业 Python IDE（集成开发环境，集成了：代码编辑、调试、测试、AI辅助等等）。

● 3.5.2 PyCharm 的主要功能

功能	对 AI 开发的意义
智能代码补全	写 <code>torch.nn.</code> 时自动列出所有可用的层和函数
智能提示与纠错	拼写错误（如 <code>traing</code> ）会被标红，第一时间发现
项目管理	多个 AI 实验项目独立管理，互不干扰
虚拟环境管理	不同 AI 项目使用不同的库版本（如项目A用 PyTorch 1.9，项目B用 PyTorch 2.0）
强大的调试器	训练出错时逐步执行，观察变量变化，快速定位问题
Git 版本控制集成	AI 实验代码可以随时提交到 GitHub，记录每一次尝试

● 3.5.3 下载并安装 PyCharm

1. 打开浏览器，访问 JetBrains 官网：
`https://www.jetbrains.com/pycharm/download/`
2. 下载 **Community Edition**（社区版，免费且开源）。
3. 双击安装包，按提示安装：
 - 建议勾选「Create Desktop Shortcut」（创建桌面快捷方式）
 - 建议勾选「Add `bin` folder to the PATH」（将 PyCharm 添加到命令行）
 - 其他选项默认即可。

● 3.5.4 创建新项目并选择 Conda 环境

1. 打开 PyCharm，点击 **New Project**。
2. 设置项目位置（Location）：例如
`D:\PythonProjects\my_first_ai_project`。
3. 在 **Interpreter type** 部分：选择 **Custom environment**。
4. 在 **Environment** 部分：选择 **select existing**。

5. 在 **Type** 部分：选择 **Conda**。
6. 在 **Path to conda** 部分：选择 Anaconda 路径。
7. 在 **Environment** 部分：选择环境。

4. 第一个 Python 程序

4.1 新建 Python 文件

1. 打开 PyCharm，打开或创建你的项目（例如 **my_first_project**）。
2. 在项目根目录上右键 → **New** → **Python File**。
3. 文件名输入 **hello**（PyCharm 会自动加上 **.py** 后缀）。
4. 在编辑器中输入以下代码：

python

```
1 # 这是第一个 Python 程序
2 print("Hello, Python!")
3 print("欢迎来到 AI 编程世界")
```

4.2 运行程序

右击代码编辑区域，选择 **Run** 即可运行程序。

运行后，在底部的 **Run** 窗口会看到输出：

python

```
1 Hello, Python!
2 欢迎来到 AI 编程世界
3
4 Process finished with exit code 0
```

exit code 0 表示程序正常结束，没有错误。

4.3 代码解释

代码	含义
<code># 这是第一个 Python 程序</code>	单行注释，不会被计算机执行，只给人看
<code>print("...")</code>	Python 的 输出函数 ，将引号中的内容打印到屏幕上

注意： `print` 后面的括号和引号必须是**英文符号**，不能是中文括号或中文引号，否则会报错。

5. Python 的注释

注释是写给程序员看的说明，计算机执行时会忽略。写注释是良好的编程习惯。

5.1 单行注释

以 `#` 开头，直到行末都是注释。

```
python
1 # 这是一条注释
2 print("Hello") # 这也是注释，可以放在代码后面
```

5.2 多行注释

使用三个单引号 `'''` 或三个双引号 `"""` 包裹多行文字。

```
python
1 '''
2 这是多行注释
3 可以写很多行
4 '''
5 print("OK")
```

多行注释实际上是一个没有被赋值给变量的字符串，Python 解释器会读取但不会执行，效果等同于注释。

6. Python 的缩进规则

这是 Python 与其他语言（如 C、Java）最明显的区别之一。Python 使用缩进来表示代码块，而不是花括号 `{}`。

6.1 什么是缩进？

缩进是指代码行开头的空格或制表符（Tab）。在 Python 中，同一级别的代码必须具有相同的缩进量。

```
python
1  # 正确示例
2  if 5 > 3:
3      print("5大于3")    # 缩进 4 个空格
4      print("条件成立")  # 同样的缩进
5  print("无论条件如何都会执行") # 没有缩进，表示与 if 无关
```

6.2 缩进错误的后果

如果缩进不一致，Python 会报 `IndentationError`。

```
python
1  if 5 > 3:
2      print("正确")
3      print("错误")    # 缩进不一致（少了一个空格）
```

运行时会报错：

```
python
1  IndentationError: unexpected indent
```


6.3 缩进规范

- **强烈建议使用 4 个空格** 作为一级缩进。
- 不要混用 Tab 和空格。PyCharm 会自动将 Tab 转换为空格。
- 冒号 `:` 后面紧跟着的下一行必须缩进（表示代码块的开始）。



为什么 Python 要强制缩进？

强制缩进让代码结构一目了然，减少了因花括号缺失导致的错误，也迫使程序员写出更整洁的代码。

7 Python 的执行机制

7.1 Python 是解释型语言

计算机只能理解二进制的机器码。我们写的 Python 代码，需要通过另一个程序（**解释器**）来逐行翻译成机器码并执行。

- **编译型语言**（如 C、C++）：先一次性将整个代码翻译成机器码（编译），再运行。执行速度快，但修改后需要重新编译。
- **解释型语言**（如 Python、JavaScript）：边翻译边执行，不需要提前编译。开发效率高，便于调试。

7.2 Python 程序的执行过程

虽然 Python 被称为“解释型”，但它内部其实有一个隐式的“编译”步骤：

1. **源码**（`.py` 文件）由 Python 解释器读取。
2. 解释器将源码 **编译成字节码**（一种中间形式，存在内存中或生成为 `.pyc` 文件）。
3. Python 虚拟机（PVM）逐条执行字节码。



对初学者来说，你只需要知道：写好的 `.py` 文件直接交给 Python 命令就能运行，不用关心编译步骤。

7.3 两种运行方式

方式	命令 / 操作	特点
脚本模式	<code>python hello.py</code> 或 PyCharm 运行	一次性执行整个文件，适合正式程序
交互式模式	命令行输入 <code>python</code> 进入 <code>>>></code> 提示符	输入一行执行一行，适合快速测试语法

第二章：变量与基础数据类型

1. 变量基础

1.1 什么是变量？

生活类比：变量就像一个贴着标签的盒子。你可以在盒子里放任何东西（数据），然后通过标签（变量名）找到它。你也可以更换盒子里的内容，标签不变。

在 Python 中，变量是内存中数据的 **名字** 或 **引用**。执行 `a = 10` 时，Python 做了三件事：

- 1. 在内存中开辟空间，存储数字 `10`；
- 2. 创建变量名 `a`；
- 3. 让变量名 `a` 指向这块内存区域。

```
python
1 | a = 10          # a 指向整数 10
2 | print(a)        # 输出 10
```

1.2 变量命名规则（必须遵守）

规则	说明	正确示例	错误示例
只能包含字母、数字、下划线	不能有空格或特殊符号	<code>my_age</code>	<code>my-age</code>
不能以数字开头	首字符必须是字母或下划线	<code>age2</code>	<code>2age</code>
区分大小写	<code>Name</code> 和 <code>name</code> 不同	<code>score</code>	-
不能使用 Python 关键字	关键字有特殊含义	<code>my_class</code>	<code>class</code>



Python 关键字: `if`, `for`, `while`, `def`, `class`, `import`, `True`, `False`, `None` 等。

1.3 命名规范（建议遵守）

类型	命名风格	示例
变量名	小写字母 + 下划线 (snake_case)	<code>student_name</code>
常量（不期望改变）	全大写 + 下划线	<code>MAX_HEIGHT = 200</code>
类名	首字母大写 (PascalCase)	<code>StudentInfo</code>

2. Python 的动态类型

动态类型 意味着变量本身没有固定类型，它的类型取决于当前指向的值，且可以随时改变。



python

```
1  x = 10          # x 是整数
2  print(type(x))  # <class 'int'>
3
4  x = "Hello"     # x 变成字符串
5  print(type(x))  # <class 'str'>
```

```
6
7 x = 3.14          # x 变成浮点数
8 print(type(x))    # <class 'float'>
```



type() 函数：用于查看变量或数据的类型。

优点：灵活，代码简洁。

缺点：大型项目中可能因类型错误引发不易发现的 bug（可通过类型注解缓解）。

3. 基础数据类型

3.1 数值类型：int 和 float

类型	说明	示例
int	整数（可正可负，无小数）	-5 , 0 , 100
float	浮点数（小数）	3.14 , -0.5 , 2.0

● 算术运算符：

运算符	名称	示例	结果
+	加法	3 + 2	5
-	减法	5 - 1	4
*	乘法	2 * 3	6
/	除法（结果总是浮点数）	7 / 2	3.5
//	整除（向下取整）	7 // 2	3
%	取余（模运算）	7 % 2	1
**	幂运算	2 ** 3	8




```

1  a = 10
2  b = 3
3  print(a + b)    # 13
4  print(a - b)    # 7
5  print(a * b)    # 30
6  print(a / b)    # 3.3333333333333335
7  print(a // b)   # 3
8  print(a % b)    # 1
9  print(a ** b)   # 1000

```



注意：/ 结果永远为 `float`；// 为向下取整（负数时注意：`-7 // 3` 结果为 `-3`）。

3.2 字符串（`str`）

字符串是 **文本数据**，用引号括起来。支持单引号、双引号、三引号。

```

python
1  s1 = 'Hello'
2  s2 = "Python"
3  s3 = """多行
4  字符串"""

```

● 常用操作

操作	说明	示例	结果
+	拼接	"Hello" + " " + "World"	"Hello World"
*	重复	"Hi" * 3	"HiHiHi"
len()	长度	len("Python")	6
str()	转换为字符串	str(123)	"123"

● 转义字符

使用反斜杠 `\` 表示特殊字符。

转义序列	含义
<code>\n</code>	换行
<code>\t</code>	制表符 (Tab)
<code>\\</code>	反斜杠本身
<code>\'</code>	单引号
<code>\"</code>	双引号

python

```
1 print("Hello\nWorld")
2 # 输出:
3 # Hello
4 # World
```

● 3.2.1 字符串的索引和切片

字符串中的每个字符都有一个**索引**（位置编号），从 `0` 开始。

python

```
1 s = "Python"
2 # 索引: P y t h o n
3 #      0 1 2 3 4 5
4 #     -6 -5 -4 -3 -2 -1
```

获取单个字符（索引）：

python

```
1 print(s[0])    # 'P'（第一个字符）
2 print(s[3])    # 'h'（第四个字符）
3 print(s[-1])   # 'n'（最后一个字符，负索引从右向左）
```

4 | `print(s[-3])` # 'h' (倒数第三个字符)

切片 (获取子字符串): 语法 字符串[start:end:step]

- **start** : 开始索引 (包含)
- **end** : 结束索引 (不包含)
- **step** : 步长 (默认为1)



python

```
1 s = "Python"
2 print(s[0:4]) # 'Pyth' (索引0,1,2,3, 不包括4)
3 print(s[2:]) # 'thon' (从索引2到末尾)
4 print(s[:4]) # 'Pyth' (从开头到索引3)
5 print(s[::2]) # 'Pto' (步长2, 取索引0,2,4)
6 print(s[::-1]) # 'nohtyP' (步长为负, 反转字符串)
```



切片不会修改原字符串, 而是返回一个新的字符串。

● 3.2.2 字符串常用方法

方法	说明	示例	结果
<code>upper()</code>	全部转为大写	<code>"Hello".upper()</code>	<code>"HELLO"</code>
<code>lower()</code>	全部转为小写	<code>"Hello".lower()</code>	<code>"hello"</code>
<code>strip()</code>	去除两端空白字符（空格、换行等）	<code>" hi ".strip()</code>	<code>"hi"</code>
<code>replace(old, new)</code>	替换子串	<code>"hello".replace("l", "x")</code>	<code>"hexxo"</code>
<code>split(sep)</code>	按分隔符拆分为列表	<code>"a,b,c".split(",")</code>	<code>['a', 'b', 'c']</code>
<code>join(iterable)</code>	用字符串连接列表元素	<code>"-".join(["a", "b"])</code>	<code>"a-b"</code>
<code>find(sub)</code>	查找子串第一次出现的索引，找不到返回 -1	<code>"hello".find("ll")</code>	<code>2</code>
<code>count(sub)</code>	统计子串出现次数	<code>"ababa".count("aba")</code>	<code>1</code> （不重叠）
<code>startswith(prefix)</code>	是否以某前缀开头	<code>"Python".startswith("Py")</code>	<code>True</code>
<code>endswith(suffix)</code>	是否以某后缀结尾	<code>"Python".endswith("on")</code>	<code>True</code>
<code>isdigit()</code>	是否全是数字	<code>"123".isdigit()</code>	<code>True</code>
<code>isalpha()</code>	是否全是字母	<code>"abc".isalpha()</code>	<code>True</code>

```
python
1 s = " Hello Python World "
2 print(s.strip())           # "Hello Python World"
3 print(s.upper())           # " HELLO PYTHON WORLD "
4 print(s.replace("Python", "Java")) # " Hello Java World "
5 print(s.split())           # ['Hello', 'Python', 'World']
   （默认按空白拆分）
6 print(", ".join(["a", "b"])) # "a,b"
```



```
7 | print(s.find("Python"))    # 7
8 | print(s.startswith(" He"))# True
9 | print("123".isdigit())     # True
```

❗ Important

注意：字符串是**不可变对象**，以上所有方法都不会修改原字符串，而是返回一个新的字符串。

3.3 布尔类型 (`bool`)

布尔值只有两个：`True` 和 `False`（首字母大写）。常用于条件判断。

```
python
1 | is_student = True
2 | has_graduated = False
```

● 比较运算符（结果均为布尔值）

运算符	含义	示例	结果
<code>==</code>	等于	<code>5 == 3</code>	<code>False</code>
<code>!=</code>	不等于	<code>5 != 3</code>	<code>True</code>
<code>></code>	大于	<code>5 > 3</code>	<code>True</code>
<code><</code>	小于	<code>5 < 3</code>	<code>False</code>
<code>>=</code>	大于等于	<code>5 >= 3</code>	<code>True</code>
<code><=</code>	小于等于	<code>5 <= 3</code>	<code>False</code>

● 逻辑运算符

运算符	含义	规则
and	与	两边都为真，结果才为真
or	或	至少一边为真，结果为真
not	非	取反

python

```
1 a = True
2 b = False
3 print(a and b)    # False
4 print(a or b)     # True
5 print(not a)      # False
```

4. 输入与输出

4.1 输出: print()

python

```
1 print("Hello, world!")
2 print("姓名:", name, "年龄:", age)    # 多个值自动加空格
```

自定义分隔符和结尾:

python

```
1 print("a", "b", "c", sep="-")    # a-b-c
2 print("Hello", end="")           # 不换行
3 print("World")                    # HelloWorld
```

4.2 输入: `input()`

`input()` 等待用户输入, 返回 **字符串**。

```
python
1 name = input("请输入你的名字: ")
2 print("你好, " + name)
```



重要: 即使输入数字, 返回的也是字符串类型, 需用 `int()` 或 `float()` 转换后才能做数值运算。

4.3 格式化输出

- 方式一: `+` 拼接 (不推荐, 数字需手动转字符串)

```
python
1 age = 20
2 print("我今年" + str(age) + "岁")
```

- 方式二: `%` 格式化 (旧式)

```
python
1 name = "小明"
2 score = 95
3 print("学生: %s, 成绩: %d" % (name, score))
4 # %s 字符串, %d 整数, %.2f 浮点数保留两位小数
```

- 方式三: f-string (Python 3.6+, **最推荐**)

```
python
1 name = "小明"
```

```
2 score = 95
3 print(f"学生: {name}, 成绩: {score}")
4 print(f"圆周率约等于: {3.1415926:.2f}")
5 print(f"{name}明年{age+1}岁") # 花括号内可写表达式
```

5. 类型转换

函数	作用	示例
<code>int(x)</code>	转换为整数	<code>int("123")</code> → 123
<code>float(x)</code>	转换为浮点数	<code>float("3.14")</code> → 3.14
<code>str(x)</code>	转换为字符串	<code>str(100)</code> → "100"
<code>bool(x)</code>	转换为布尔值	<code>bool(1)</code> → True

陷阱与解决:

```
python
1 int("3.14") # ❌ ValueError: invalid literal for int()
2 # 正确做法:
3 int(float("3.14")) # 3
```

自动转换 (隐式): 不同类型混合运算时, Python 自动提升类型。

```
python
1 result = 5 + 3.2 # int + float → float, 结果为 8.2
```

6. 实战案例: BMI 计算器

根据用户身高 (米) 和体重 (公斤) 计算 BMI:

$$BMI = \frac{\text{体重}(kg)}{\text{身高}(m)^2}$$

```
python
```



```
1 # 输入身高（米）和体重（公斤）
2 height = float(input("请输入身高(m): "))
3 weight = float(input("请输入体重(kg): "))
4
5 # 计算 BMI
6 bmi = weight / (height ** 2)
7
8 # 输出结果，保留两位小数
9 print(f"您的BMI指数为: {bmi:.2f}")
```

7. 课堂练习

1. 圆的面积与周长

输入半径，输出面积（ πr^2 ）与周长（ $2\pi r$ ）， $\pi = 3.14159$ 。

2. 平均值计算器

输入两个整数，输出它们的平均值（保留两位小数）。

3. 自我介绍生成器

输入姓名和年龄，输出：“我叫XX，今年XX岁，很高兴认识你！”

4. 进阶挑战

输入摄氏温度，转换为华氏温度：

$$F = C \times \frac{9}{5} + 32$$

第三章：流程控制

1. 流程控制概述

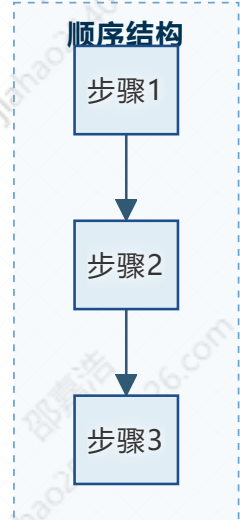
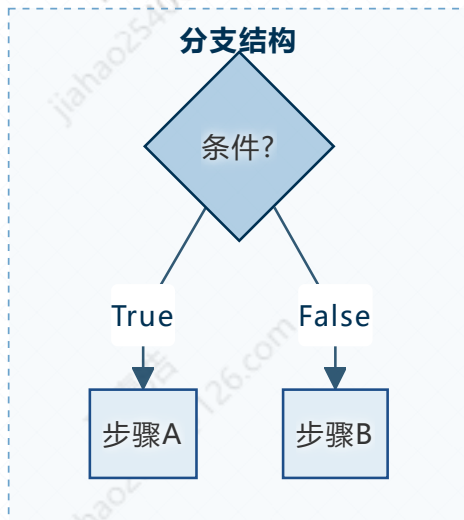
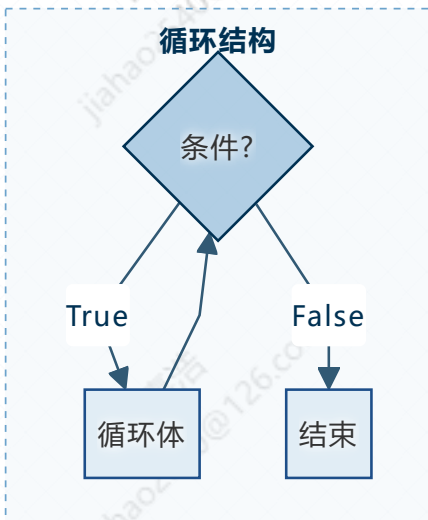
定义：程序默认是**从上到下顺序执行**的。流程控制语句可以改变这种顺序，让程序根据条件选择执行不同的代码块，或者重复执行某段代码。

三种基本结构：

结构	作用	关键词
顺序结构	代码按书写顺序依次执行	无特殊关键词
分支结构	根据条件选择执行某一段代码	<code>if</code> , <code>elif</code> , <code>else</code>
循环结构	重复执行某一段代码	<code>while</code> , <code>for</code>

生活类比：

- 顺序结构：起床 → 刷牙 → 吃早餐 → 出门。
- 分支结构：如果下雨 → 带伞；否则 → 不带伞。
- 循环结构：重复练习 10 次投篮。



2. 条件判断（分支结构）

2.1 单分支： `if`

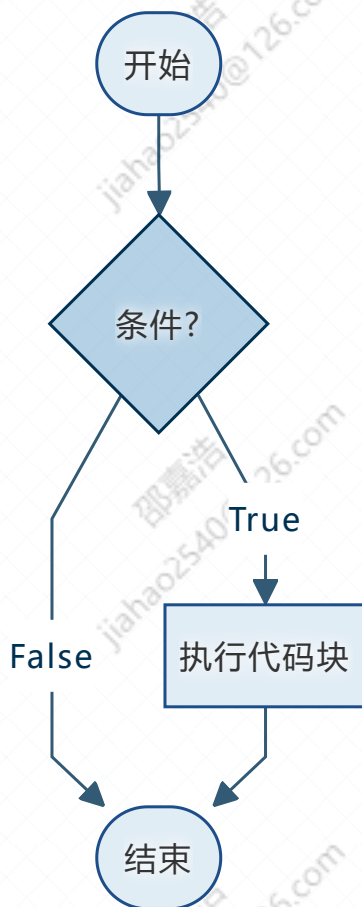
语法：

```
python
1  if 条件:
2      条件成立时执行的代码块
```

- 条件 是一个布尔表达式（结果为 `True` 或 `False`）。

- 代码块必须**缩进**（通常 4 个空格）。

流程图：



示例：



python

```
1 age = 20
2 if age >= 18:
3     print("已成年，可以进入")
```

2.2 双分支：if-else

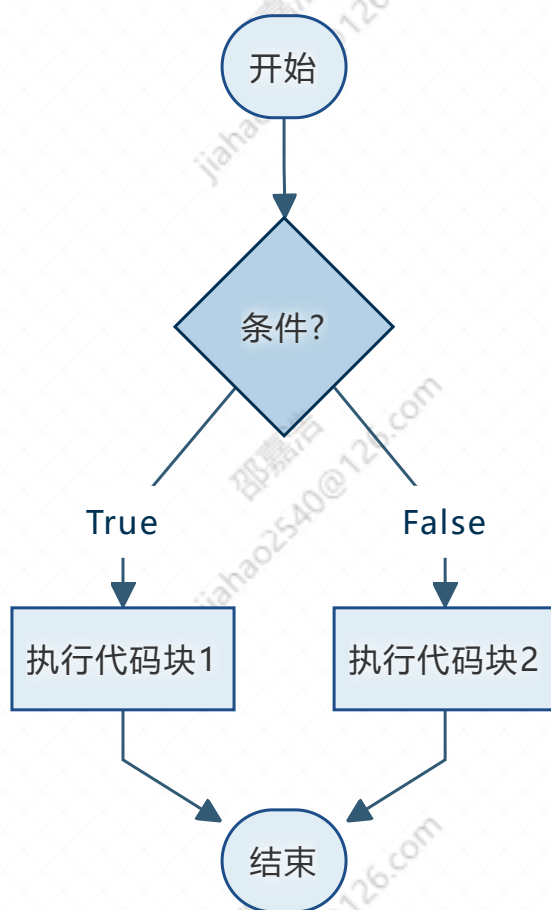
语法：



python

```
1 if 条件:
2     条件成立时执行
3 else:
```

流程图：



示例：



python

```
1 num = 7
2 if num % 2 == 0:
3     print("偶数")
4 else:
5     print("奇数")
```

2.3 多分支：if-elif-else

语法：



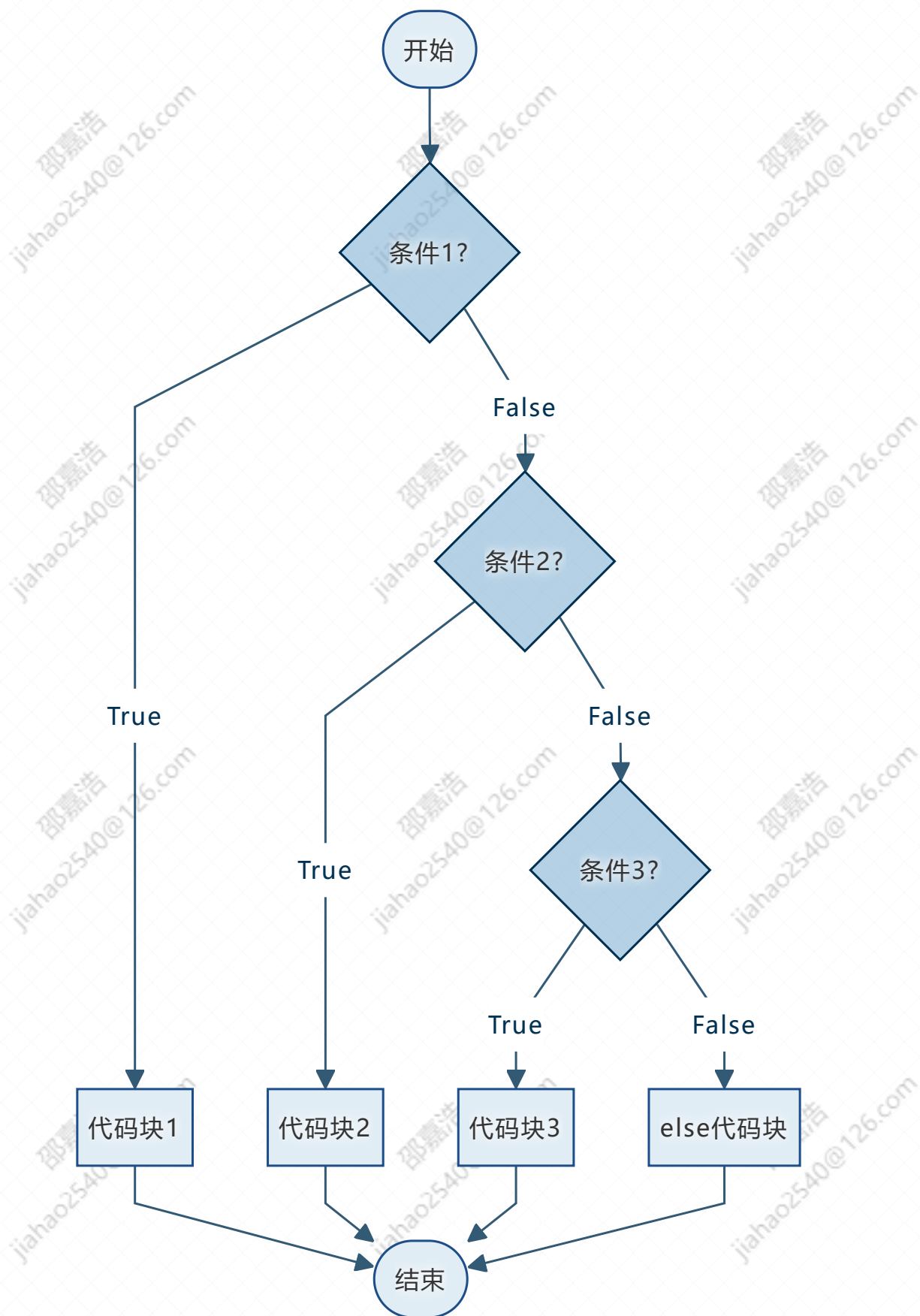
python

```
1 if 条件1:
```

```
2      代码块1
3  elif 条件2:
4      代码块2
5  elif 条件3:
6      代码块3
7  ...
8  else:
9      代码块n
```

- `elif` 可以有多个, `else` 可选。
- 从上往下依次判断, 遇到第一个 `True` 就执行对应代码块, 然后跳出整个结构。

流程图:



示例：成绩等级评定



python

```
1 score = 85
2 if score >= 90:
```

```
3     grade = "A"
4     elif score >= 80:
5         grade = "B"
6     elif score >= 70:
7         grade = "C"
8     elif score >= 60:
9         grade = "D"
10    else:
11        grade = "F"
12    print(f"成绩等级: {grade}")    # B
```

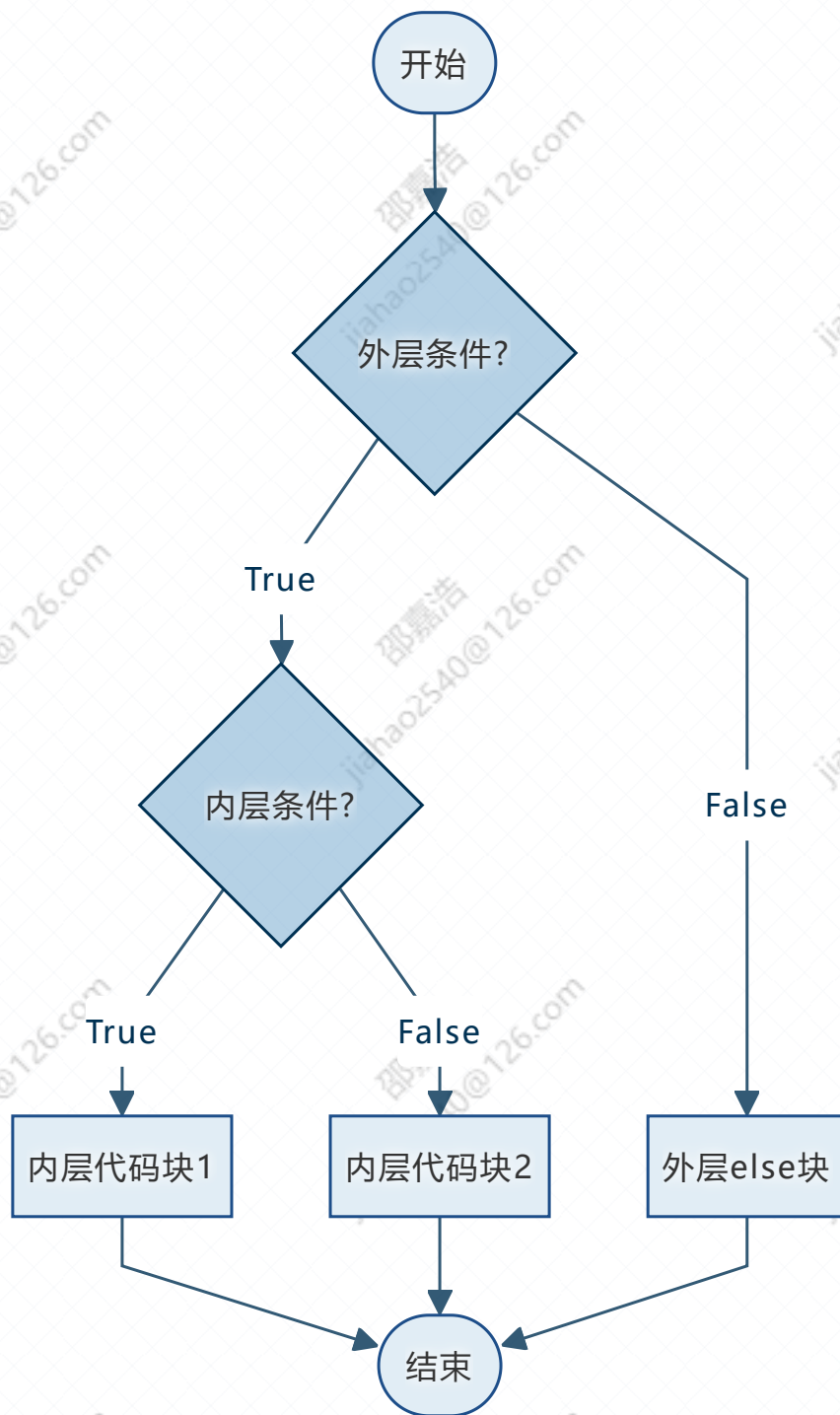
💡 Tip

注意顺序：条件应该由苛刻到宽松，否则可能导致逻辑错误。例如把 `score >= 80` 放在 `score >= 90` 前面，90 分会先匹配 B。

2.4 嵌套分支

在 `if` 或 `else` 的代码块内部再写一个完整的 `if` 语句。

流程图：



示例：影院入场

python

```
1 age = 20
2 has_ticket = True
3
4 if age >= 18:
5     if has_ticket:
6         print("可以进入影院")
7     else:
```

```
8         print("请先购票")
9     else:
10         print("未成年人禁止入内")
```

2.5 条件表达式（三元运算符）

语法： 值1 if 条件 else 值2

当条件为 `True` 时返回 值1，否则返回 值2。适用于简单的二选一赋值。

```
python
1 age = 18
2 status = "成年" if age >= 18 else "未成年"
3 print(status) # 成年
```

2.6 课堂练习

1. 奇偶判断

编写程序，输入一个整数，如果它是偶数，输出 "偶数"；如果是奇数，输出 "奇数"。

2. 比较大小

输入两个整数 `a` 和 `b`，输出较大的那个数（假设两个数不相等）。

3. 季节判断

输入一个整数表示月份（1~12），输出对应的季节：

- 3~5 月 → "春季"
- 6~8 月 → "夏季"
- 9~11 月 → "秋季"
- 12、1、2 月 → "冬季"

（提示：可以使用 `or` 逻辑运算符连接多个条件）

3. 循环结构

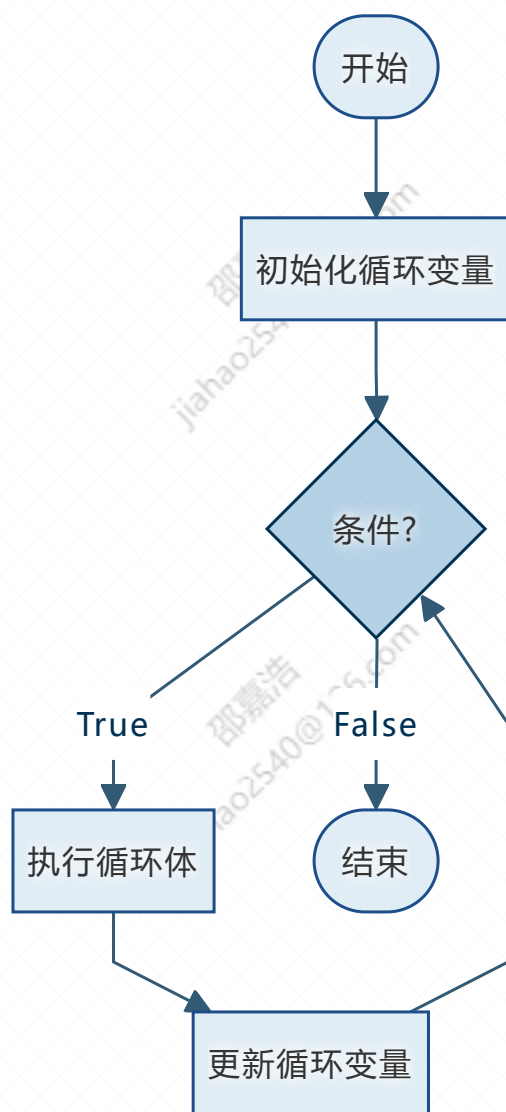
3.1 while 循环

语法：

```
python
1 while 条件:
2     循环体
```

- 每次循环前检查条件，为 **True** 则执行循环体，为 **False** 则退出。
- 循环体内通常需要修改条件相关的变量，否则形成**死循环**。

流程图：



示例：打印 1 到 100


```
1 i = 1
2 while i <= 100:
3     print(i)
4     i += 1
```

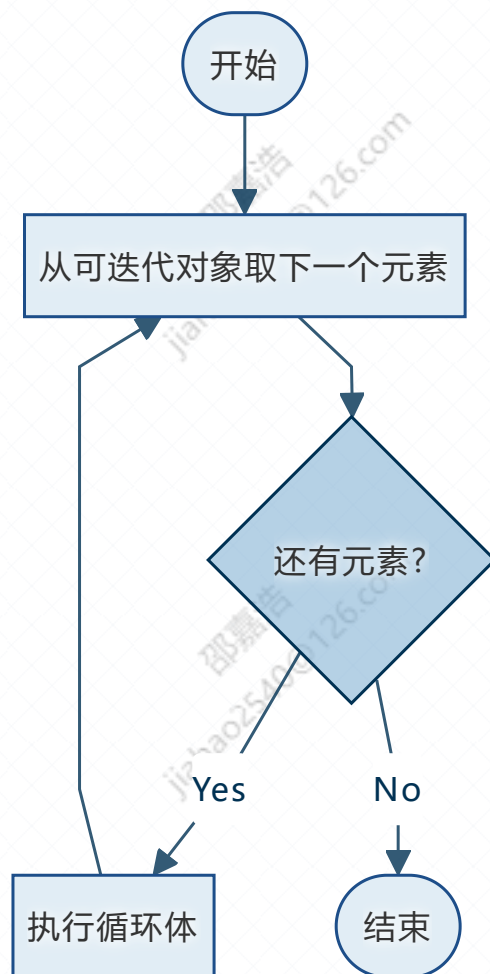
3.2 for 循环

语法：

```
1 for 变量 in 可迭代对象:
2     循环体
```

- 依次从可迭代对象（字符串、列表、`range` 等）中取出元素，赋值给变量，执行循环体。
- 遍历完成后自动退出，不需要手动维护索引。

流程图：



常用 `range(stop)` 和 `range(start, stop, step)` :

调用方式	生成序列	示例
<code>range(stop)</code>	0 ~ stop-1	<code>list(range(5))</code> → <code>[0,1,2,3,4]</code>
<code>range(start, stop)</code>	start ~ stop-1	<code>list(range(2,6))</code> → <code>[2,3,4,5]</code>
<code>range(start, stop, step)</code>	start, start+step, ...	<code>list(range(1,10,2))</code> → <code>[1,3,5,7,9]</code>

python

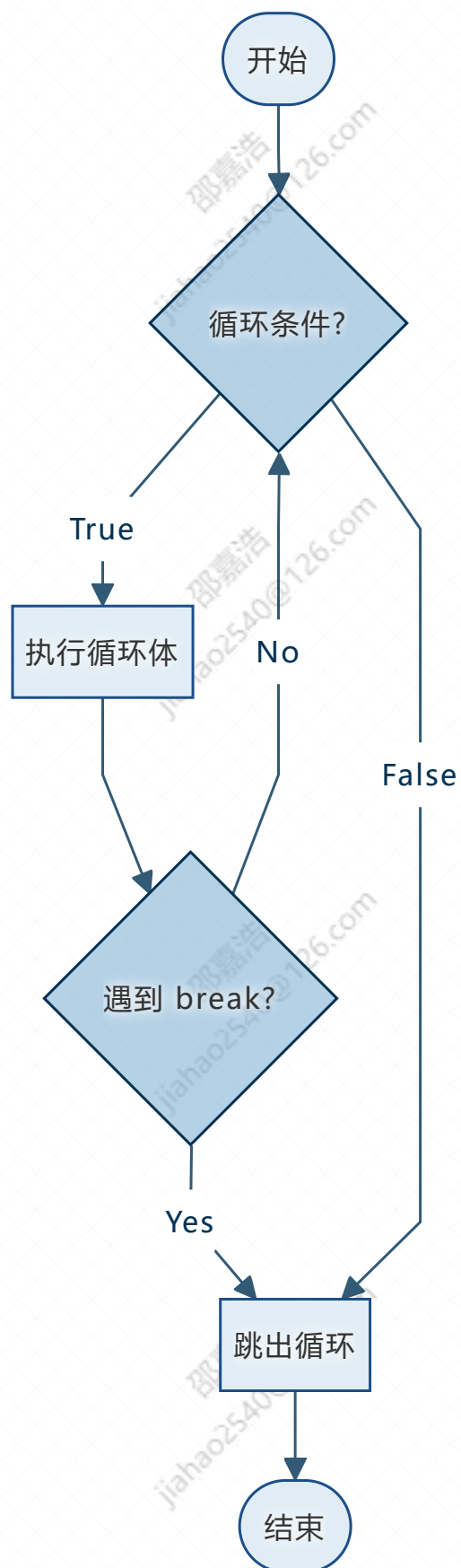
```
1 # 遍历 0 到 4
2 for i in range(5):
3     print(i)
4
5 # 遍历字符串
6 for ch in "Python":
7     print(ch)
8
```

```
9 # 遍历列表（第4章会详细讲）
10 fruits = ["苹果", "香蕉", "橙子"]
11 for fruit in fruits:
12     print(fruit)
```

4. 循环控制语句

4.1 **break**：终止整个循环

流程图：



python

```
1 for i in range(10):  
2     if i == 5:  
3         break
```

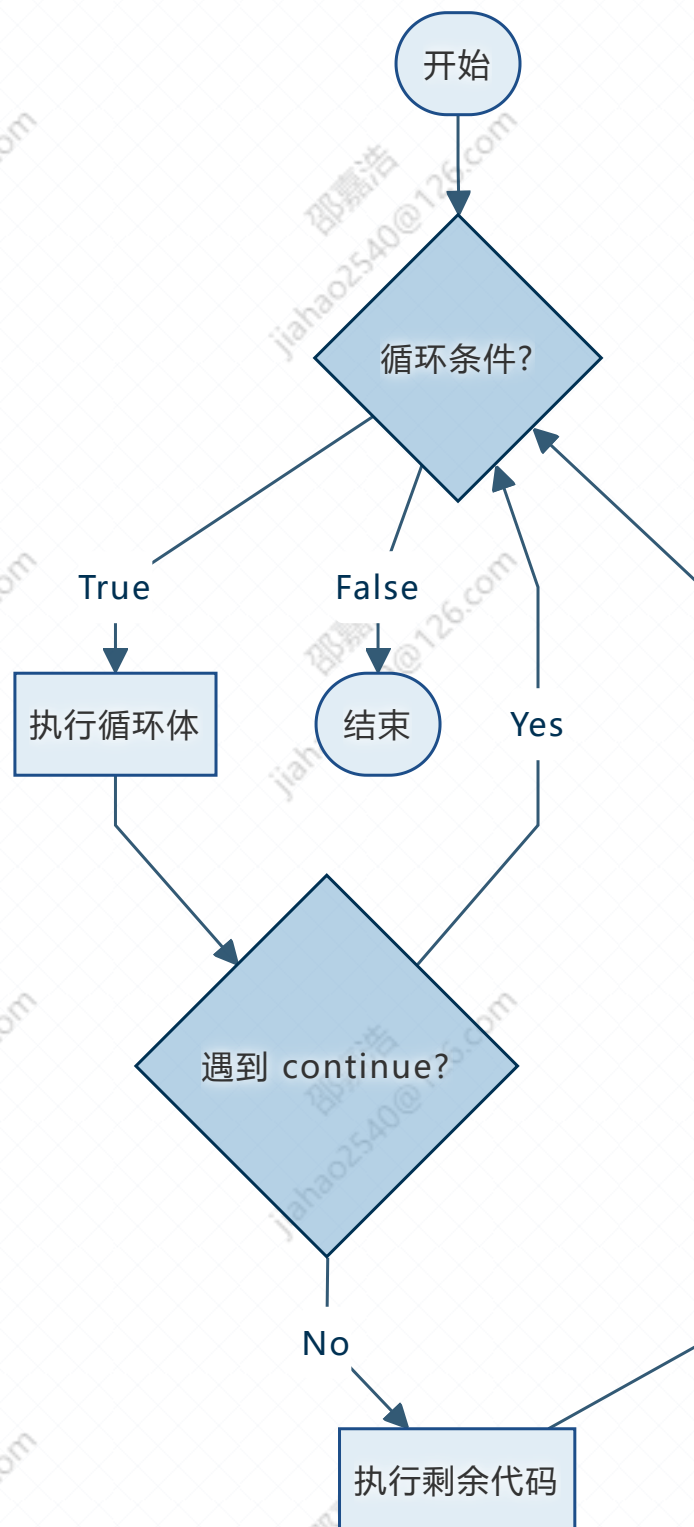
4 | `print(i)` # 输出 0,1,2,3,4

④ Note

`break` 只影响它所在的**最内层**循环。

4.2 `continue`：跳过本次循环，进入下一次

流程图：



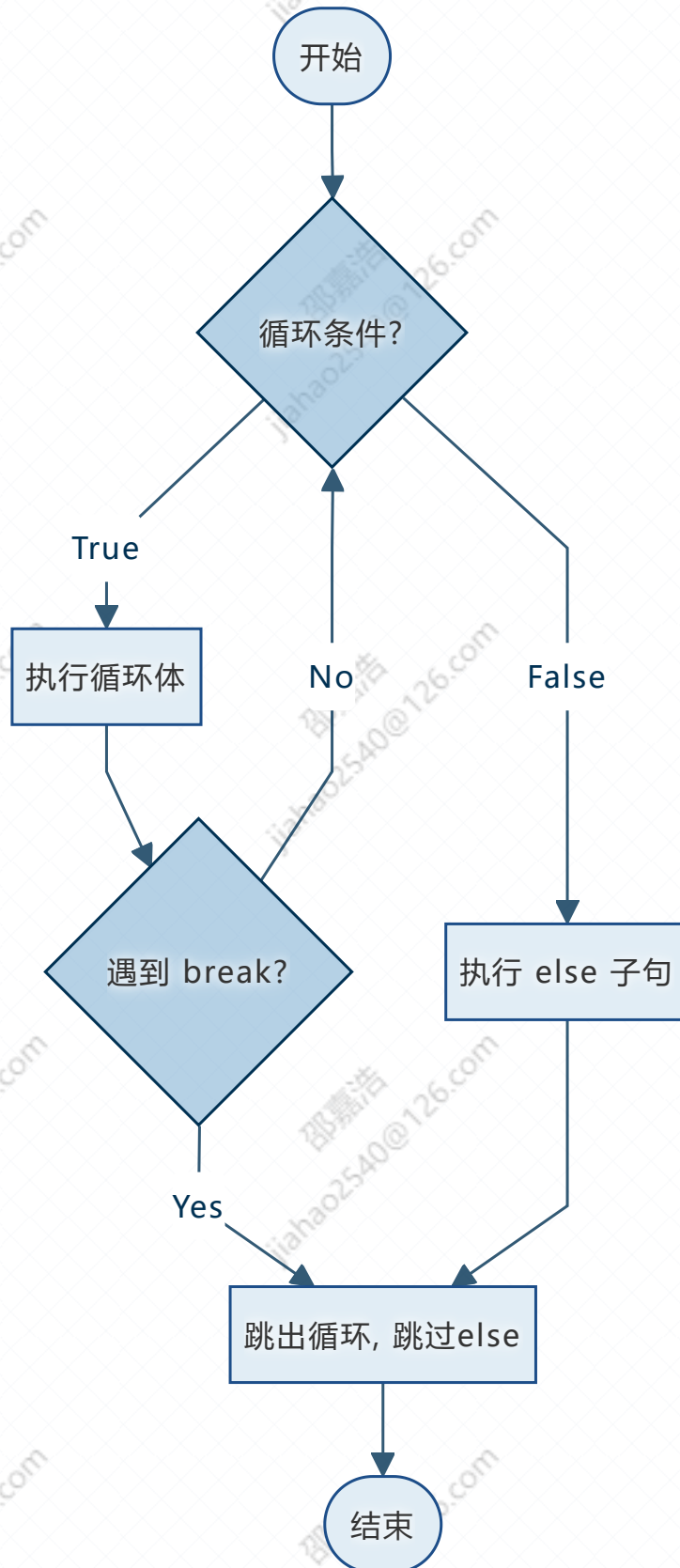
python

```
1  for i in range(5):
2      if i == 2:
3          continue
4      print(i)    # 输出 0,1,3,4
```

4.3 else 子句（用于循环）

定义：while 或 for 循环可以带一个 else 子句，当循环正常结束（没有被 break 中断）时，执行 else 中的代码。

流程图：



```
1 # 示例：判断是否为质数
2 num = 17
3 for i in range(2, num):
4     if num % i == 0:
5         print(f"{num} 不是质数")
6         break
7 else:
8     print(f"{num} 是质数") # 循环没有被 break 时执行
```

如果循环被 `break` 打断，`else` 不会执行。

4.4 `pass` 空语句

作用：占位，什么也不做。当语法上需要一条语句但逻辑上不需要任何操作时使用。

```
1 if age < 18:
2     pass # 暂时不处理未成年人
3 else:
4     print("欢迎")
```

5. 实战案例：猜数字游戏

程序随机生成一个 1~100 的整数，用户循环猜测，提示“大了”“小了”，猜中后输出猜测次数并结束。

```
1 import random
2
3 # 随机生成目标数字
4 target = random.randint(1, 100)
5 guess_count = 0
6
```

```

7  print("猜数字游戏开始！数字范围 1~100")
8
9  while True:
10     guess = int(input("请输入你猜的数字: "))
11     guess_count += 1
12
13     if guess == target:
14         print(f"恭喜你，猜中了！共用了 {guess_count} 次")
15         break
16     elif guess > target:
17         print("太大了，再试试")
18     else:
19         print("太小了，再试试")

```



此案例用到了 `import random`（导入随机数模块）、`while True` 死循环配合 `break` 退出、`if-elif-else` 分支判断。

6. 课堂练习

- 奇数偶数判断**：输入一个整数，输出它是奇数还是偶数。
- 累加器**：用 `while` 循环计算 $1+2+3+\dots+100$ 的值。
- 遍历字符串**：用 `for` 循环遍历字符串 `"Python"`，依次输出每个字符。
- 寻找第一个偶数**：给定列表 `[7, 9, 12, 15]`（可以模拟），用 `for` 循环找出第一个偶数并打印，找到后立即终止循环。
- 打印乘法口诀表**：用嵌套 `for` 循环输出 9×9 乘法表（不需要对齐，只需输出 `i*j=...` 即可）。

